Econometrics Softwares

Session 2: Data cleaning in R

Mathilde Col & Jérémy Do Nascimento Miguel

University of Bordeaux, France

Recap of the last session

- R: the programming language, the interpreter software, the console
- RStudio: the IDE for console, scripts, environment, graphs and more
- Organizing a project
- Packages: installing and loading
- Types and operators
- Data structures: vectors, matrices and data frames
- Importing data in R
- Subsetting variables and observations

This session

Basic cleaning and econometrics using real-life data in R:

- Cleaning data with the dplyr package
- Introduction to regex
- Functions & Loops
- Merging and append two datasets
- Reshaping dataset
- Descriptive statistics

Data cleaning with dplyr

Sorting data

- With base R, you can use the order function

```
mydatas3[order(mydatas3$num_deaths), ]
```

- Use the **arrange()** function from **dplyr** to order based on values of specific variable

```
arrange(mydatas3, num_deaths)
```

- The default is in ascending order (older observations at the top here)
- For descending order you can use a minus in front of the variable

```
mydatas3[order(-mydatas3$num_deaths), ]
arrange(mydatas3, -num_deaths)
```

Modifying data

With **dplyr** you have the **mutate()** function

```
mydatas4 <- mydatas3 %>%
    mutate(
    initial_pop = total_population + num_deaths
    )
```

The **mutate()** function lets you do multiple modification at once

```
mydatas4 <- mydatas3 %>%
    mutate(
    initial_pop = total_population + num_deaths,
    mort_rate = num_deaths / total_population
    )
```

Modifying subset of data

In the case of **dplyr**, you need to combine **mutate()** with the **case_when()** function

```
mydatas4 <- mydatas3 %>%
    mutate(
        any_death = case_when(
        num_deaths == 0 ~ "No death",
        num_deaths > 0 ~ "Some deaths",
        .default = NA
    )
)
```

You can also combine **mutate()** with the function **ifelse()** (from base R) if there is only one condition:

```
mydatas4 <- mydatas3 %>%
    mutate(
        any_death = ifelse(num_deaths > 0, "Some deaths", "No deaths")
    )
```

Modifying by group

You can use the **group_by()** function:

```
mydatas3_grouped <- mydatas3 %>%
    group_by(state) %>%
    mutate(state_mort_rate = sum(num_deaths)/sum(total_population))
```

- This gives the same value for all observations of each state
- To ungroup the data you need to use the **ungroup()** function
- Since dplyr 1.1.0 in 2023, the .by option has be added

```
mydatas3_grouped <- mydatas3 %>%
    mutate(state_mort_rate = sum(num_deaths)/sum(total_population),
            .by = state)
```

Data type constraints

Data type	Example	R data type
Text	Country, City, Name, Address	character
Integer	Population, labor force, number of rooms	integer, numeric
Decimal	Temperature, GDP	numeric, double
Binary/Dummy	Woman/man, Married/Single	logical, factor
Category	Marriage status, time zone	factor
Date	Date of birth, Order dates	Date

- Each of these data types is treated differently
- We have to ensure that every variable in our dataset is the correct data type

Checking data types

Logical checking : returns TRUE or FALSE

- is.character(), is.numeric(), is.logical(), is.factor(), is.Date(), ...

When writing the code you do some checks manually with **table()** or **summary()**:

- the age is above 0 and below 100
- the birth date is before the death date
- the provided countries' name are in the list of valid countries names
- the sum of sub groups is equal to the total
- the average value is close to official statistics
- ...

Transforming variables

- We can use the **as.*()** functions to transform a variable from one type to another

```
mydatas3$cases <- as.numeric(mydatas3$cases)
class(mydatas3$case)
# [1] "numeric"
```

- Now we can estimate the average number of cases

```
mean(mydatas3$cases)
# [1] 40.52027
```

Cleaning strings

We will see how to use stringi package and its simplified version, the stringr package.

First manipulation: to lower, UPPER, Capitalize the string (Stringr documentation)

```
library(stringr)
str_to_lower("Hello world")
# [1] "hello world"
str_to_upper("Hello world")
# [1] "HELLO WORLD"
str_to_title("Hello world")
# [1] "Hello World"
```

Comma problems

- Let's examine the variable *percent_below_poverty*

```
class(mydatas3$percent_below_poverty)
# [1] "character"
head(mydatas3$percent_below_poverty)
# [1] "9,3" "9,3" "9,3" "9,3" "9,3" "9,3"
```

- We have commas instead of dots to specify decimals

```
as.numeric(mydatas3$percent_below_poverty)
```

- R will replace the observations including commas with NA

A bit of regular expressions

- We need to replace the commas with dots, BEFORE transforming to numeric
- The **stringr** package is a simplified wrapper of the **stringi** package

```
library(stringr)
str_replace(mydatas3$percent_below_poverty, ",", ".")
# str_replace_all() will replace ALL occurences
```

- We can now replace the string by a number

```
mydatas3$percent_below_poverty <- mydatas3$percent_below_poverty |>
    str_replace(",", ".") |>
    as.numeric()
```

More regular expressions

Find and replace patterns in strings

```
str_detect(c("apple", "banana", "pear"), pattern = "an")
# [1] FALSE TRUE FALSE
```

- . : any character
- ? : previous character is optional
- + : at least one of the previous character
- * : 0 or more of the previous character
- {a, b} : between a and b of the previous character
- [...] : any of the characters between brackets
- | : one pattern or another
- ^ : start of string
- $: end of string
- \: escape special characters

```
str_extract("Born the 01/01/2000 in Addis", "[0-9]{2}/[0-9]{2}/[0-9]{4}")
# [1] "01/01/2000"
```

String encoding

- In a computer everything is stored in 8 bits: a number between 0 and 255
- encoding: each number is associated with a value
- The most basic one is the ASCII encoding, covering most of the characters
- Now we mainly use the UTF-8 for most documents
- You might encounter or require other encodings with some files, especially the ones with other alphabets like Chinese, Japanese, Cyrillic or Arabic
- When reading or writing a file, you generally have an *encoding* option
- Useful trick to remove accents with the **stringi** package: **stri**$_t$*rans*$_g$*eneral*

Dropping duplicates

- Duplicates are a common issue in raw survey data
- We can use the **distinct()** function from **dplyr**

```
data_unique <- distinct(mydatas3)
sum(duplicated(data_unique))
```

- *data_unique* indeed includes no duplicates

Missing values

- Ideally registered as NA
- But often reported as "NA", -1, 0, 8, 99, ... in data
- You can use the **is.na()** function to find NA

```
is.na(miss_rain)
```

- You can count them by summing the result with the **sum()** function

```
sum(is.na(miss_rain))
# [1] 100849
```

The most basic imputation is to replace a missing value by the mean, by using **replace_na()** function from the **tidyr** package

```
data_replace_3 <- data_unique |>
    mutate(rain_filled = replace_na(rain, mean(rain, na.rm = TRUE)))
```

Exercise 1

- Open the dataset called "hh_characteristics.xlsx" and store it under the name you want
- Clean the variable "sex" so that it takes the value 0 for men and 1 for women
- Create the variable "educ" from the variable "edyears" so that it corresponds to the category:
  - 0 = No education
  - 1 = Primary but not completed
  - 2 = Primary completed
  - 3 = Secondary but not completed
  - 4 = Secondary completed
  - 5 = Tertiary
- Clean the woreda name (admin3name_en)

Functions & Loops

Functions

- You can write your own function, store it in an object and reuse it latter
- A function is composed of the **arguments** and the **body/code** of the function

```
my_function <- function(arguments) {
    # R code
}
```

- A simple example

```
add <- function(a, b) {
    c <- a + b
    return(c)
}
add(2, 5)
# [1] 7
```

Loops: For loops

- Loops are very important programming construct
- As the name suggests, in a loop, the program repeatedly do a set of instructions until some condition is met
- The syntax of a **for** loop is

```
for (x in list) {
    # R code to run
}
```

For loops: example 1

Looping over a vector

```
fruits <- c("mangos", "bananas", "kiwis")
for (fruit in fruits) {
    # The paste() function concatenates strings together
    print(
        paste("I like", fruit)
    )
}
# [1] "I like mangos"
# [1] "I like bananas"
# [1] "I like kiwis"
```

The **paste()** function is a basic function to concatenate strings

For loops: example 2

Nested loops

```
for (i in 2:3) {
    for (fruit in fruits) {
        print(
            paste("Can I get", i, fruit, "please?")
        )
    }
}
# [1] "Can I get 2 mangos please?"
# [1] "Can I get 2 bananas please?"
# [1] "Can I get 2 kiwis please?"
# [1] "Can I get 3 mangos please?"
# [1] "Can I get 3 bananas please?"
# [1] "Can I get 3 kiwis please?"
```

Loops: apply functions

- Another way to do functions is the **apply()** family of functions
- They *apply* a given function to all the elements of a list
- The most used is **lapply()** which returns a list of results

```
lapply(list, function)
```

- A short example

```
# Apply the square root function to all numbers in 1 to 10
lapply(1:10, sqrt)
```

- You can add parameters to a function, or define a short function that takes a parameter

```
# Apply the sprintf() function on 1 to 10, with 2 digits and a leading 0
lapply(1:10, sprintf, fmt = "%02d")
# Apply a function that takes a parameter x and returns x / (1 + x)
# To all numbers in 1 to 10
lapply(1:10, function(x) x / (1 + x))
```

Reshaping data

Pivot table: wide to long

The syntax of **pivot_longer()** is based on 4 elements

- data: the data to pivot
- cols: the list of columns that contain values to group
- names_to: the new column to create to store the names of these columns corresponding to the values
- values_to: the new column to create to store the values from these columns

Pivot table: wide to long example

- Using the World Bank population data from the **tidyr** package
- You can access World Bank data with the **wbstats** package

```
data("world_bank_pop")
world_bank_pop
```

We have:

- One row per country/indicator
- One column per year
- Cells are the values

We first want:

- One row per country/year/indicator
- One new column to identify the year
- One new column with the values

```
world_bank_pop_long <- world_bank_pop |>
  pivot_longer(
    cols = `2000`:`2017`,
    names_to = "year",
    values_to = "value"
  )
```

Pivot table: long to wide

The syntax of **pivot_wider()** is based on 3 elements

- data: the data to pivot
- names_from: the column with the names used as new column names
- values_from: the column with the values to fill these new columns

Pivot table: long to wide example

- Using the long version of the World Bank data
- The structure of this data is:
  - One line per country, year and indicator
  - One column with all the values
  - Cells are the values
- We want
  - One row per country/year
  - One column to identify country/year
  - One column per indicator

```
world_bank_pop_clean <- world_bank_pop_long |>
  pivot_wider(
    names_from = "indicator",
    values_from = "value"
  )
```

- The example in the documentation is a bit different, I let you check

Combining databases

- You generally use multiple databases that you combine
- Appending databases with the same columns is simple with **rbind()**

```
mydatas3_rows_01_10 <- mydatas3[1:10, ]
mydatas3_rows_20_30 <- mydatas3[20:30, ]
rbind(mydatas3_rows_01_10, mydatas3_rows_20_30)
```

- Merging databases with the same rows is also simple with **cbind()**

```
mydatas3_cols_1_2 <- mydatas3[, 1:2]
mydatas3_cols_3_4 <- mydatas3[, 3:4]
cbind(mydatas3_cols_1_2, mydatas3_cols_3_4)
```

- However, you generally don't have perfectly matching data to merge
- You want to merge the datasets based on their relations

Joining data in R

- In R, this can be done with **base** R or the **dplyr** package
- R 4 Data Science chapter on joins goes into details using **dplyr**
- With **base** R you can use the **merge()** function
- We will use the group of databases on flights from the **nycflights13** package
  - **airlines**: the list of airlines identified by a carrier code (*carrier*)
  - **airports**: the list of airports identified by three letter airport code (*faa*)
  - **planes**: the list of planes identified by their tail number (*tailnum*)
  - **weather**: weather records in airports identified by their airport and time (*origin* and *time_hour*)
  - **flights**: list of flights and their characteristics

```
data("airlines", package = "nycflights13")
data("airports", package = "nycflights13")
data("planes", package = "nycflights13")
data("weather", package = "nycflights13")
data("flights", package = "nycflights13")
```

Flight data summary

session_3/img/relational.png

- We keep some columns of the data

```
flights2 <- select(flights, year, time_hour, origin, dest,
    tailnum, carrier)
```

Joining data types

- Different types of merging/joining:
  - **left join**: keep all the values on the first table and add missing values in second table when no match found
  - **right join**: keep all the values on the second table and add missing values in first table when no match found
  - **inner join**: keep only the values for keys found in both tables
  - **outer join**: keep all values and add missing values for both sides when needed
- In **base** R you can use the **merge()** function to do an inner join by default, with the **all.x** and **all.y** options to chose if you want to keep mismatch on either datasets
- With **dplyr** you can use the **left_join()**, **right_join()**, **inner_join()** and **full_join()** functions

Joining data examples

- With the **merge()** function

```
merge(flights2, airlines)
```

- With the **left_join()** function

```
left_join(flights2, airlines)
# Joining with `by = join_by(carrier)`
# A tibble: 336,776  7...
```

  - **left_join()** is more explicit, faster and tells you which column is used
  - Ideally, you should specify which column you use with the **by** option

```
merge(flights2, airlines, by = "carrier")
left_join(flights2, airlines, by = join_by(carrier))
```

- If the columns have different names, you can use the **by.x** and **by.y** options for **merge()** or the == in **join_by()** function for **left_join()**

Exercice 2

Use the dataset from Exercise 1:

- You want 1 household = 1 observation, how do you do?
  - Identify the unique identifier per observation/line in the long form
  - Change the data from long to wide
- You want to merge the dataset obtained with the geographical dataset used in Session 1:
  - Open the geographical dataset with the population information
  - Identify the common variable between the two and check its compatiblity in the two datasets
  - Merge the two and store it in a new dataset

Descriptive statistics

Basic statistics for data analysis

What we usually check before starting the regression analysis:

- Sample size
- Average values: mean, median (overall and for groups)
- Spread of data: min, max, SD
- Distribution: percentiles, skewness, histogram
  - **quantile()** to get the quantile, **percent_rank(x)** to get in which percentile is the observation, **cum_dist()** to get the cumulative distribution of a variable
- Frequency of values
  - We use **table()** to have the frequency and values for each variable

Basic statistics for data analysis

- Number of observations (i.e. rows) and variables (i.e. columns)

```r
nrow(mydata)
ncol(mydata)
```

- Basic summary statistics for all or specified variables

```r
# For all variables in the data
summary(mydata)
# For variable tracking only
summary(mydata$tracking)
```

- Two-way frequency tabulations

```r
table(mydata$tracking, mydata$etpteacher)
```

Aggregated statistics

If your data is in long format, you can use **group_by(*grouping variable*)** or **.by = "*grouping variable*"**:

```
dat1 <- df_long %>%
    mutate(mean_gender = mean(gender, na.rm = T), .by = hhid)
dat1 <- df_long %>% group_by(hhid) %>%
    mutate(mean_gender = mean(gender, na.rm = T))
```

If your data in is wide format, you can use **rowMeans()**:

```
dat1 <- df_wide %>%
    mutate(mean_all = rowMeans(.),
        mean_sel = rowMeans(.[select_vars]))
```

Other useful commands:

- **n()** counts the number of observation
- **sum()** makes the sum of a variable

Exercise 3

Use the dataset produced in Exercise 2:

- OPTIONAL BUT WILL MAKE THE REST OF THE EXERCISES EASIER: Change the data from wide to long again
- What is the share of women in the sample?
- What is the average household size?
- What is the share of the population that got interviewed in each woreda (per thousand)

Home task

If you want to practice, and you want us to look at your script, please do the exercises 1 to 3, and download it to the Drive folder (link below, but will also be sent by email)

- It must be a "*.R" file, as we saw during the class
- Name it "NAME_Firstname_session_1.R", and put on this Drive folder